Where Does a Wrong Step Hurt? Error Position, Remaining Steps, and Thinking Mode in Multi-Hop Reasoning

[TBD: Authors]
Dept. of CSE (AI&ML), M.S. Ramaiah Institute of Technology, Bengaluru, India

***Abstract*—When a language model makes a wrong step in a chain of reasoning, does the position of that step decide whether the model recovers? Prior studies disagree, and their designs confound position with the number of steps that remain and with the final, answer-producing step. We plant controlled errors at exact hop positions in models' own reasoning chains on 3- and 4-hop MuSiQue questions, resume generation from the corrupted chain, and measure recovery relative to an uncorrupted resume of the same chain. We test Qwen3-8B with thinking off and on (same weights) and Llama-3.1-8B, using two error types and 384–922 eligible items per model and mode. Late intermediate errors are recovered 9–26 percentage points less often than early ones, with this late-stage fragility appearing in every model and mode. An error in the final hop is almost never recovered without thinking (at most 15%), so pooling it with "late" errors would overstate the position effect. For Qwen3 the effect follows the number of steps left after the error, not position itself: at a fixed number of remaining steps, position has no effect. Thinking raises recovery (0.82 vs. 0.58 at intermediate hops) but does not change the position effect, and almost all recoveries are silent. We also find that prefilling a hop summary inside the thinking block ends deliberation, a pitfall for prefill-based studies of reasoning models. [TBD: one sentence on think_trace if it works.]**

***Index Terms*—chain-of-thought, error propagation, multi-hop question answering, reasoning models, robustness**

I. Introduction

Language models answer multi-step questions by chaining intermediate results, and any step can go wrong. Whether a wrong step ruins the answer depends on whether the model later notices or works around it. If errors at some positions are reliably recovered and errors at others are not, step-level verification [1] and checks in agent pipelines should concentrate where recovery fails. Knowing *where* a wrong step hurts most therefore has practical value.

Prior studies disagree on the answer. Von Recum et al. [2] intervene in reasoning models' own chains at fixed points and find that early interventions hurt more. Jia and Mu [3] perturb one step and let the model continue. They find that error propagation falls from 33.5% for early steps to 22.5% for late ones, but call the gradient "partly mechanical": earlier steps have more steps left after them, and task difficulty dominates. ASCoT [4] reports the opposite, *late-stage fragility*, for Qwen2.5-7B on GSM8K. Its largest accuracy drops, however, come from corrupting the final step (52 points at step 4 of 4), which produces the answer directly. These designs leave three variables entangled: the position of the error, the number of steps that remain after it, and whether the corrupted step is the terminal one. All three studies also use math, science or logic tasks, whose intermediate steps have no gold values, so an injected error's position and correctness are only approximate.

We study error position in multi-hop question answering, where these variables can be controlled. Each MuSiQue [5] question is composed of single-hop questions with gold intermediate answers. We keep items whose own chain matches the gold decomposition hop by hop, replace one hop's answer with a wrong value of the same entity type, and let the model continue from the corrupted chain. Recovery is measured against an uncorrupted resume of the same chain. Crossing chain length (3 and 4 hops) with hop position yields pairs of errors that leave the same number of steps but sit at different positions. The terminal hop is analyzed separately. Qwen3-8B [6] runs with thinking off and on using the same weights, and Llama-3.1-8B [7] serves as a second non-thinking model.

Our findings are as follows. Late intermediate errors are recovered *less* often than early ones in every model and mode, by 9 to 26 percentage points where the confidence interval excludes zero. Late-stage fragility therefore holds even with the terminal step excluded, contrary to our pre-stated hypothesis. Terminal-hop errors are almost never recovered without thinking: the models copy the wrong value into their answer. On linear chains, position and remaining steps are nearly collinear, but branching questions let them vary separately. There, for Qwen3, the effect follows the number of steps left after the error, not position itself. For Llama a position effect may remain. Thinking raises recovery substantially but leaves the position effect unchanged. Finally, recovery is almost always silent: the model writes the correct value later without remarking on the planted one.

Our contributions are: (i) to our knowledge, the first position-controlled error-injection study on multi-hop QA, with exact error positions obtained by aligning each model's own chain to the gold decomposition; (ii) a design that separates the terminal step and compares errors with matched remaining steps, showing late-stage fragility on intermediate hops; (iii) a same-weights comparison of thinking and non-thinking modes, together with a methodological caution: a hop summary prefilled inside the thinking block ends the model's deliberation; (iv) a released, resumable injection harness [TBD: repository URL].

II. Related Work

Multi-hop QA. HotpotQA [8] and 2WikiMultiHopQA [9] pose questions that require combining facts from several paragraphs, but many can be answered through shortcuts. MuSiQue [5] composes questions from single-hop questions and filters out shortcut-solvable ones, which gives gold intermediate answers for every hop. Models often answer the sub-questions correctly but fail to compose them (the compositionality gap [10]). CRiT-QA [11] swaps counterfactual entities into the reasoning chain and adds distractor paths that diverge at different hops. It tests robustness to errors in the *input*, whereas we place errors in the model's own reasoning.

Chain-of-thought and thinking modes. Chain-of-thought prompting [12] elicits intermediate steps. Reasoning models trained with reinforcement learning, such as DeepSeek-R1 [13], produce long deliberations before answering. Qwen3 [6] offers both modes in one set of weights, which lets us compare them without changing the model.

Errors and self-correction. Without external feedback, models rarely fix their own reasoning [14]. They struggle to locate errors but can often correct an error once its location is given [15]. Process supervision trains verifiers to check each step [1]. Our results bear on where such checks matter most.

Interventions on chains of thought. Editing or truncating a chain and observing the answer is a standard probe of whether the answer depends on the stated reasoning [16]. Explanations can be unfaithful to the factors that actually drive the answer [17], [18]. Yee et al. [19] find that models often reach the correct answer despite invalid reasoning text (unfaithful recovery), and that faithful and unfaithful recovery respond differently to how obvious the error is. Separately, Fragile Thoughts [20] compares perturbation types across 13 models. The three studies of error position discussed in the introduction [2]–[4] belong to this family. We add a task with exact hop positions, a separate terminal step, and matched remaining steps.

Position effects in the input. Position also matters on the input side. Models use information in the middle of long contexts less well [21], are sensitive to the order of premises [22], and are distracted by irrelevant context [23]. Our in-context error type draws on this last finding: it plants a value taken from the shown distractor paragraphs.

III. Research Questions

We stated the following questions and hypotheses before the main runs. All concern recovery from a single planted error (Section IV).

- **RQ1 (position).** Does the position of an intermediate error affect recovery? *H1:* early errors propagate more, so recovery rises with relative position (PSI > 0).
- **RQ2 (remaining steps).** Is a position effect explained by the number of steps that remain? *H2:* in contrasts with matched remaining steps, the position effect shrinks toward zero.
- **RQ3 (terminal step).** Is the answer-producing hop special? *H3:* terminal-hop errors have the lowest recovery for every model, and pooling them with late errors changes the size or sign of PSI.
- **RQ4 (thinking).** Does thinking change the level or the shape of the position curve? *H4:* with the same weights, thinking raises recovery and shrinks |PSI|.
- **RQ5 (error type).** Do plausible errors hurt more? *H5:* in-context distractors are recovered less often than off-context swaps.

A position effect is declared absent only if its 90% CI lies within ± 5 percentage points. Whether recovery is explicit or silent is analyzed as an exploratory question.

IV. Method

A. Task and data

We use MuSiQue-Ans v1.0 [5], whose questions are composed from single-hop questions, each with a gold intermediate answer and a supporting paragraph. We keep answerable questions with $K \in \{3, 4\}$ hops from the train and dev splits (dev: 760 three-hop and 405 four-hop; train: 4,387 and 1,175). Two-hop questions are excluded because they have only one intermediate hop. Because the method is inference-only, train items are usable; we report the split of every item and compare results across splits.

MuSiQue composes hops in five shapes. In the *linear* shapes (3hop1, 4hop1) every hop consumes the previous answer. In the *branching* shapes (3hop2, 4hop2, 4hop3) some hops are independent and are merged later. We read each hop's dependencies from the `#N` references in the gold decomposition. Linear chains are our primary analysis set; all items form a robustness check.

Context. Each item is shown its gold supporting paragraphs plus four fixed non-supporting paragraphs (distinct titles, none sharing a title with a gold paragraph), sampled with a per-item seed and shuffled with the same seed, so every model and condition sees the identical context.

B. Prompt and own-chain baseline

All models receive the same instruction (Fig. 1), asking for one line per hop, `Hop N: <sub-question> -> <answer>`, followed by `Final answer: <short answer>`. Prompts are built with each model's own chat template. For each item we generate the model's *own* chain (one sample) and a *closed-book* answer (same instruction without paragraphs).

Eligibility. An item enters the injection study only if its own chain (i) gives a correct final answer (exact match after SQuAD normalization [24] against the answer and its aliases), (ii) contains exactly K hop lines numbered $1..K$, (iii) matches the gold decomposition hop by hop, in order (after removing parenthetical remarks, each hop answer has normalized exact match or token-F1 ≥ 0.8 with the gold hop answer, or contains it as whole words with at most three extra words; the final hop is also checked against the answer aliases), (iv) is not truncated, and (v) the closed-book answer is wrong, which removes items the model can answer from memory. Every failed criterion is logged per item. Requirement (iii) guarantees

```
Answer the question using only the paragraphs
below. Reason one hop at a time. This question
needs exactly {K} hops.
Write each hop on its own line as
"Hop N: <sub-question> -> <answer>", where
<answer> is a short name, date or number,
not a sentence.
After hop {K}, write "Final answer: <short answer>".

Example of the format (a 3-hop question):
Question: What is the official language of the
country whose capital is the city where the
Eiffel Tower stands?
Hop 1: In which city does the Eiffel Tower
stand? -> Paris
Hop 2: Which country has Paris as its
capital? -> France
Hop 3: What is the official language of
France? -> French
Final answer: French

Paragraphs:
Title: <title>
<paragraph text>
...
Question: <question>
```

Fig. 1. Prompt used for all models and modes (K = the item's hop count). The closed-book variant drops "using only the paragraphs below" and the paragraphs. Stating K, asking for short hop answers and showing one invented example were added after the pilot, in which the unconstrained prompt led Qwen3-8B to split questions into 5–11 hops and answer hops in full sentences, leaving almost no chain alignable with the gold decomposition.

that the hop we corrupt is the hop the gold decomposition says it is, which is what makes the error position exact.

C. Conditions

For an eligible item with K hops we run:

- **noprefix**: the original prompt, a fresh sample; it defines item difficulty.
- **clean**$_k$, $k = 1..K$: the model's own chain truncated after hop line k, then resumed.
- **inject**$_k$, $k = 1..K$, for each of two error types: as clean$_k$, but the answer on hop line k is replaced by a planted value.

This gives $1 + 3K$ cells per item (10 for K=3, 13 for K=4) before validity drops. Hop K produces the final answer; corrupting it is analyzed separately as the *terminal* step and is never pooled with "late" intermediate hops.

D. Injection by assistant-turn prefill

Corruptions are applied to the model's own text, not to a gold template. The prefix is the model's own answer text cut at the end of hop line k, with only the answer span after the arrow on that line replaced; it ends in exactly one newline. The prefix is appended after the chat template's assistant header, so the model continues its own turn: no new user turn is opened and no end-of-turn token is placed at the splice. For Qwen3 in non-thinking mode the template's empty think block precedes the prefix; in thinking mode the prefix is placed inside the think block (`<think>\n` + hop lines), and the model continues in hop format inside the think block before it answers (confirmed in the pilot). [TBD: describe the think_trace injection site if its run succeeds.]

Prompt text is tokenized once, without adding special tokens, and the identical token ids are passed to the inference engine. Unit tests check, for every tokenizer used, that the spliced ids contain no end-of-turn token after the template, that the template tokens are unchanged by the splice, and that Llama receives exactly one BOS token.

E. Error types

Every hop answer receives a type tag: regular expressions for years, five date formats and plain numbers; otherwise the spaCy (`en_core_web_sm`) [25] entity label of the answer's mention in its supporting paragraph. Candidate values are kept only when their in-context label agrees with the label of the string on its own (covering the whole string), which removes many small-model mistags.

- **In-context distractor**: a string of the same type from the item's four *shown* distractor paragraphs (titles preferred, then named entities and years). It looks supported by the context.
- **Off-context swap**: a gold hop answer of the same type from a *different* MuSiQue item that occurs nowhere in this item's question or shown paragraphs, preferring answers to the same relation (e.g., *place of birth*). It is supported by nothing in the context.

Within these pools candidates are ranked by length similarity to the original answer, with a seeded tie-break per item and hop.

F. Validity rules

A planted value is used only if all hold, otherwise the next candidate is tried; cells with no valid candidate are dropped and logged with the failing rule: (1) after normalization it is not equal to, a substring of, or a superstring of any gold answer, alias, or gold hop answer of the item; (2) its type tag equals the original's; (3) its format matches (year vs. year, digits vs. none, casing style, trailing punctuation); (4) it does not already occur in the model's own baseline output. We audited 100 planted values (25 per error type and K) for type match and plausibility. The two raters were one author and an LLM (Claude Sonnet), each rating independently. The type rule works on coarse tags, so many planted values are the wrong kind of thing within a tag: a state for a city (both GPE), or a TV network for a person mistagged as ORG. The LLM rater judged 50% of the sample to be the right fine-grained type and 41% plausible as an answer to the sub-question. Rates were similar at early and late hops (plausible: 41% vs. 38%), high for PERSON and date values, and lowest for ORG (18% plausible). The author was stricter on type (43% right type) and similar on plausibility (42%). The raters agreed on 91% of type judgments (Cohen's $\kappa = 0.82$) and 79% of plausibility judgments ($\kappa = 0.57$). Only 42% of values were the right type for both raters, and 31% plausible for both. Our errors

TABLE I
DECODING SETTINGS (ONE SAMPLE PER CELL).

	T	top-p	top-k	max new tokens
Qwen3-8B, non-thinking	0.7	0.8	20	1024
Qwen3-8B, thinking	0.6	0.95	20	4096
Llama-3.1-8B-Instruct	0.6	0.9	–	1024

are therefore best described as same-coarse-type substitutions, many of them implausible; Section VI-B tests whether this affects the position result.

G. Outcomes

The final answer is the text after the last `Final answer:` in the answer part of the output (after `</think>` in thinking mode); a missing final answer or a truncated run counts as wrong. **Correct** (primary) is exact match against the answer and aliases. **Adopted** is exact match against the planted value; we deliberately do not count mere mentions of the planted value, which fire on restatement. As a secondary measure for verbose answers we also report containment of the gold (resp. planted) value within the final-answer line only.

Recovery with clean-twin conditioning. Resumption itself can fail, so we measure recovery only where the uncorrupted resume succeeds:

$$\mathrm{RR}(k) = P\left(\text{inject}_k \text{ correct} \mid \text{clean}_k \text{ correct}\right). \quad (1)$$

Position sensitivity on intermediate hops is

$$\mathrm{PSI} = \mathrm{RR}_{\text{late}} - \mathrm{RR}_{\text{early}}, \quad (2)$$

with early $k{=}1$ and late $k{=}K{-}1$ ($k{=}2$ for $K{=}3$, $k{=}3$ for $K{=}4$). PSI > 0 means early errors propagate more.

Remaining steps. For hop k, *remaining* is the number of later hops that depend on it directly or transitively; on linear chains this equals $K{-}k$. Relative position is k/K. Because k + remaining $= K$ on linear chains, position, remaining steps and chain length cannot all be varied independently. We therefore compare *matched-remaining* cells, (K3,k2) vs. (K4,k3) with one step left and (K3,k1) vs. (K4,k2) with two, where absolute position differs but remaining steps are equal (the pairs also differ by one hop in K), and we include item difficulty as a covariate.

V. EXPERIMENTAL SETUP

A. Models and inference

Qwen3-8B [6], 4-bit AWQ [26] (`Qwen/Qwen3-8B-AWQ`, revision `4da05a8`), run with `enable_thinking` off and on. Same weights in both modes, so the comparison isolates the thinking mode. **Llama-3.1-8B-Instruct** [7], 4-bit AWQ (`hugging-quants/Meta-Llama-3.1-8B-Instruct-AWQ-INT4`, revision `db1f81a`), non-thinking only. Inference uses vLLM [27] 0.32.0 on one RTX 5070 (12 GB), with an 8,192-token context. Each model–mode pair runs on one GPU for all of its conditions.

Decoding (Table I) follows each model's recommended sampling settings and is identical across all conditions within a model–mode pair. Closed-book runs cap generation at 512 tokens in non-thinking mode. Each cell draws one sample with its own seed, derived from a SHA-256 hash of (run, item, condition, hop, error type), so results do not depend on batching and interrupted runs resume reproducibly. Llama's chat template adds a fixed system header ("Today Date: 26 Jul 2024"), identical for all prompts. The thinking-mode cap of 4,096 new tokens truncated 6% of pilot baselines; truncated baselines are ineligible and no pilot continuation was truncated, so we kept the cap. In the main runs, 3% of thinking baselines were truncated (and excluded), and no condition cell had more than 3% of its continuations truncated.

TABLE II
ITEMS AND CELLS PER MODEL–MODE. LINEAR: 3HOP1/4HOP1 CHAINS (PRIMARY ANALYSIS SET).

	Qwen3 non-thinking	Qwen3 thinking	Llama-3.1 non-thinking
Items attempted	6,727	2,180	6,727
Final answer correct	44%	52%	36%
Eligible ($K{=}3$ / $K{=}4$)	855 / 67	167 / 217	523 / 41
Eligible linear ($K{=}3$ / $K{=}4$)	763 / 33	147 / 154	438 / 15
Inject cells	5,413	2,565	3,289
Inject cells dropped (validity)	253	173	177
Clean twins correct	97%	79%	88%

B. Item counts

Table II gives the yield. Non-thinking runs attempt all 6,727 three- and four-hop items. The thinking run attempts 600 three-hop items and all 1,580 four-hop items, oversampling four-hop items because of their low pilot yield. Most items fail eligibility because the final answer is wrong or the chain takes a route other than MuSiQue's decomposition. Four-hop items rarely pass without thinking, so four-hop non-thinking contrasts are underpowered, far below the 300–400 items per K targeted by the power analysis.

C. Statistical analysis

All confidence intervals are item-clustered percentile bootstraps (items resampled with replacement, 2,000 replicates). We report:

- RR per model $\times$ mode $\times K \times k\times$ error type, terminal hop in separate rows;
- PSI per model $\times$ mode $\times$ error type $\times K$ with 95% CIs; a position effect is declared *absent* if the 90% CI lies within ± 5 percentage points (equivalent to two one-sided tests at $\alpha = 0.05$);
- exact McNemar tests on paired early vs. late outcomes of the same item (both clean twins correct), per model $\times$ mode $\times$ error type $\times K$, Holm-adjusted across these tests;
- matched-remaining contrasts on linear chains (items resampled independently within each K);
- terminal vs. pooled intermediate recovery;

- one logistic regression on intermediate inject cells with a correct clean twin,

$$\text{correct} \sim \text{rel_pos} \times \text{mode} + \text{remaining} + K + \text{error_type} + \text{difficulty}\,(+\,\text{model}),$$

with standard errors clustered by item and variance inflation factors reported, since rel_pos, remaining and K are partly collinear by construction.

Primary results use linear chains; all-item results and results excluding truncated runs are reported as robustness checks, since a truncated run counts as wrong and truncation could differ by position. Before the main runs we ran a pilot of 50 dev items per mode and required clean-twin accuracy of about 90% (lower would mean resumption itself is broken). In the pilot, clean twins were correct in 7/7 non-thinking and 31/32 thinking cells. In the main runs the thinking mode fell below this target (Section VI).

VI. Results

Unless stated otherwise, numbers are for linear chains, RR is conditioned on a correct clean twin, and brackets give 95% item-bootstrap CIs. All-item results agree in sign and size with every linear result below. Truncation is below 3% in every cell, and excluding truncated runs changes no PSI by more than 0.003. Four-hop non-thinking sets are small (33 linear items for Qwen3, 15 for Llama), so their contrasts have wide intervals.

A. Resumption without an error (clean twins)

Resuming from the model's own uncorrupted prefix succeeds at 96–100% of intermediate hops for Qwen3 without thinking and 82–87% for Llama (K=3). For Qwen3 with thinking it succeeds at only 68–86% (terminal hop: 100%), below the ≈90% target set before the runs. An audit of all failing twins found no scoring error: the gold answer never appears as a final answer. Instead, in thinking mode 80% of failing twins leave the model's own chain at a later hop and 21% give the right entity at a different granularity ("1991" for *December 25, 1991*); the other two runs show similar proportions. The cause in thinking mode is the prefix itself. With hop lines placed inside the think block, Qwen3 treats reasoning as finished: all 985 intermediate clean continuations contain only hop lines and a final answer, with a median of 110 generated tokens against 1,231 for an unprefixed resample. The prefill also costs accuracy in thinking mode (clean twin wrong and resample of the same item right in 183 cells vs. the reverse in 68; McNemar $p = 2 \times 10^{-13}$) but not without thinking ($p = 0.22$ for Qwen3). Thinking-mode RR is therefore measured on items the model can finish without further deliberation, and its continuations are short (Section VI-E).

B. RQ1: Position of an intermediate error

Late intermediate errors are recovered *less* often than early ones, the opposite of H1 (Fig. 2, Table III). PSI is negative in all 12 model × mode × error type ×K cells, and its CI excludes zero in 8; the four exceptions are the small four-hop non-thinking sets and one thinking cell whose CI reaches zero. The effect is clearest for Qwen3 without thinking at K=3, where recovery drops from 0.62 to 0.48 (in-context) and from 0.70 to 0.50 (off-context). In paired items, recovery at the early hop but not the late hop occurs 148 and 195 times, against 59 and 53 for the reverse (Holm-adjusted McNemar $p < 10^{-8}$). Llama shows the same pattern at K=3 (PSI $= -0.09$ and -0.13; McNemar $p_{\text{Holm}} = 0.001$ for off-context). With thinking, PSI is -0.09 to -0.12, but its paired tests do not survive Holm adjustment (98–113 pairs per cell). No cell meets the ±5 pp equivalence criterion. The direction matches the late-stage fragility reported by ASCoT [4] and contradicts the early-error account [2], [3].

Robustness to implausible planted values. Because many planted values are implausible (Section IV), we recomputed PSI separately by the type tag of the corrupted hop. Late-stage fragility is *strongest* where the planted values are cleanest. For PERSON, date and number values, Qwen3 without thinking at K=3 falls from 0.61 to 0.31 (in-context) and from 0.73 to 0.36 (off-context), and Llama's PSI is -0.14 and -0.25. For places, PSI is -0.14 to -0.21 for both non-thinking models at K=3. For ORG and NORP values, where the audit found the most implausible substitutions, PSI is near zero (CIs include 0). Implausible values thus dilute the position effect rather than produce it. The tag groups differ in which hops and items they cover (PERSON hops are mostly first hops), so we also tested the main contrast directly. The LLM rater judged a random sample of 200 cells from Qwen3 without thinking at K=3 (100 early, 100 late; outcomes hidden). Restricted to the 82 cells whose planted value was type-correct and plausible, recovery falls from 0.68 at the early hop to 0.37 at the late hop (PSI $= -0.31$ [-0.52, -0.11]), against -0.25 [-0.38, -0.11] for all 200 cells. Implausible values are recovered somewhat more often at both hops (0.77 and 0.55).

C. RQ2: Position or remaining steps?

On linear chains position and remaining steps move together, and the joint regression on those chains cannot split the effect between them (Table IV; variance inflation factors 87 and 104). Three analyses separate them. First, in branching items some hops at different positions have the same number of dependent later hops: in 3hop2, hops 1 and 2 both feed only hop 3. Within the same item, recovery at the later of two such positions minus the earlier one is $+0.05$ [-0.02, 0.12] for Qwen3 without thinking (114 items) and -0.01 [-0.10, 0.08] with thinking (42 items). Llama, however, still recovers less at the later position (-0.12 [-0.22, -0.02], 69 items). Second, the matched-remaining contrasts on linear chains (same steps left, different absolute position) give differences of -0.04 to $+0.05$ for Qwen3 without thinking, although the four-hop cells hold only about 32 items. Third, logistic models on all items that control for chain shape favour remaining steps over relative position for Qwen3 without thinking (ΔAIC = 13.6). In the joint model, remaining steps are significant ($\beta = 0.75$, $p < 0.001$) and relative position is not ($p = 0.76$). For Llama

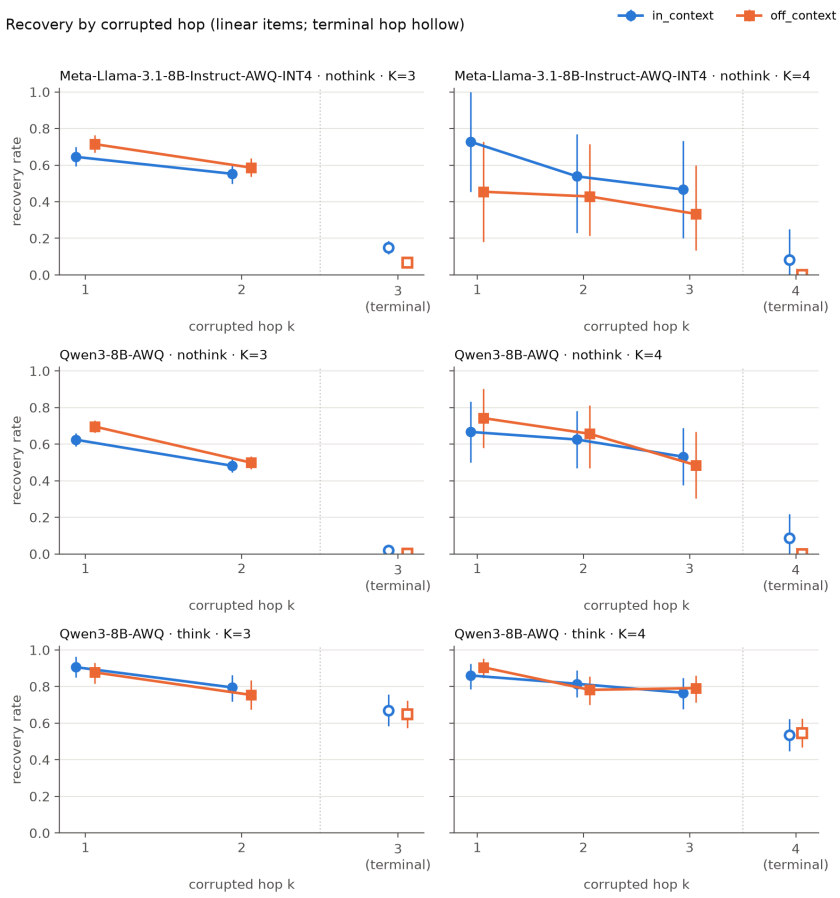


Fig. 2. Recovery rate by corrupted hop k (linear chains; 95% item-bootstrap CIs). Hollow markers: terminal hop, analyzed separately. Recovery falls from early to late intermediate hops in every panel and collapses at the terminal hop without thinking.

TABLE III
POSITION SENSITIVITY PSI $=$ $\mathrm{RR}_{\mathrm{LATE}} - \mathrm{RR}_{\mathrm{EARLY}}$ ON INTERMEDIATE HOPS (LINEAR CHAINS, 95% ITEM-BOOTSTRAP CIS). n: ELIGIBLE LINEAR ITEMS.

	K	n	in-context	off-context
Qwen3, non-thinking	3	763	-0.14 [-0.18, -0.10]	-0.20 [-0.24, -0.16]
	4	33	-0.14 [-0.34, 0.07]	-0.26 [-0.47, -0.04]
Qwen3, thinking	3	147	-0.11 [-0.19, -0.04]	-0.12 [-0.21, -0.04]
	4	154	-0.09 [-0.19, 0.00]	-0.11 [-0.20, -0.02]
Llama-3.1, non-thinking	3	438	-0.09 [-0.15, -0.03]	-0.13 [-0.19, -0.07]
	4	15	-0.26 [-0.53, 0.00]	-0.12 [-0.39, 0.12]

and for Qwen3 with thinking, the two fit equally well (ΔAIC < 2). For Qwen3, then, late-stage fragility follows the number of steps left after the error, not position itself. For Llama, the evidence leaves room for a position effect of its own.

D. RQ3: The terminal step

Errors at the terminal hop are almost never recovered without thinking: RR is 0.02 and 0.00 for Qwen3 (in-/off-context) and 0.15 and 0.07 for Llama, against 0.56–0.64 at intermediate hops of the same models. The models copy the corrupted last-hop answer into the final answer (Qwen3 adopts the planted value in 98–99% of terminal cells, Llama in 75–88%). With thinking, terminal recovery is 0.60 for both error types, still 0.22 below intermediate hops (CIs -0.29 to -0.15). Near-zero recovery here is expected, since the last hop's value is the answer and the model only has to copy it. Its importance lies in what pooling would do. Counting the terminal hop as "late", as designs that do not separate it effectively do, would multiply the apparent position effect by 2.3 to 3.2. For Qwen3 without thinking at $K{=}3$, PSI would be -0.36 and -0.45 instead of -0.14 and -0.20; for Llama, -0.30 and -0.40 instead of -0.09 and -0.13. ASCoT's largest accuracy drops came from exactly this step [4].

TABLE IV
LOGISTIC REGRESSION OF RECOVERY ON INTERMEDIATE INJECT CELLS WITH A CORRECT CLEAN TWIN (LINEAR CHAINS; 5,670 CELLS, 1,177 ITEMS; ITEM-CLUSTERED SES). VIF: VARIANCE INFLATION FACTOR.

Term	β	SE	p	VIF
rel_pos	−3.67	1.62	0.023	87
remaining	−0.58	0.52	0.27	104
K	0.23	0.30	0.44	11
mode = thinking	1.52	0.32	<0.001	13
rel_pos × thinking	−0.24	0.54	0.66	14
off-context error	0.15	0.05	0.002	1.0
item difficulty	−1.12	0.16	<0.001	1.1
model = Qwen3	−0.35	0.09	<0.001	1.2

E. RQ4: Thinking mode

Thinking raises recovery at every hop: intermediate RR is 0.83 and 0.82 with thinking against 0.56 and 0.60 without (in-/off-context; mode $\beta = +1.52$, $p < 0.001$). It does not change the shape: the relative position × mode interaction is small and not significant ($\beta = -0.24$, $p = 0.66$), and late errors remain more harmful with thinking (Table III). Because eligibility differs between modes, we repeated the comparison on the 83 linear items eligible in both (63 three-hop, 20 four-hop). There, thinking still raises recovery on the same cells (intermediate +0.24 [0.17, 0.31]; terminal +0.70 [0.63, 0.78]), and the terminal hop stays below intermediate hops in both modes. Without thinking PSI stays negative (K=3: −0.14 and −0.28), but with thinking it shrinks to 0.00 and −0.10 with CIs that include zero. The interaction remains non-significant ($p = 0.34$) on this smaller set, so the shared items neither confirm nor rule out a change of shape.

These comparisons concern a thinking mode in which the model barely deliberates after the error (Section VI-A): most recoveries happen when the model restates the chain after `</think>`, rewriting the corrupted hop with the correct entity. [TBD: think_trace: error planted mid-thought in the model's own thinking trace; continuation length, clean-twin accuracy, RR and PSI vs. the summary-prefill condition. If continuations stay short, state the summary-prefill result as "thinking format without post-error deliberation" and move this to Limitations.]

F. RQ5: Error type

Off-context swaps, which nothing in the context supports, are recovered slightly more often than in-context distractors ($\beta = +0.15$, $p = 0.002$; e.g., Qwen3 without thinking at K=3, k=1: 0.70 vs. 0.62). The difference is small next to the effects of position, the terminal hop and thinking. With thinking it is inconsistent in sign, and at Llama's terminal hop it reverses (0.15 in-context vs. 0.07 off-context).

G. How recovery happens

Recovery is almost always silent. A rule-based labeler calls a recovery *explicit* if a doubt or correction marker (e.g., "wait", "actually", "incorrect", "should be") occurs in the same or the next sentence as the planted or the true hop value, and *silent* otherwise. It marks none of Qwen3's non-thinking recoveries and 0.2% of its thinking recoveries as explicit; Llama flags the error in 9% of its intermediate-hop recoveries. Typically the model simply writes the correct entity in a later hop or in its restated chain, without commenting on the planted value. Doubt markers do not indicate success: Llama uses them more often when it fails (15%) than when it recovers (12%). These labels are rule-based and were not validated by hand beyond spot checks.

VII. LIMITATIONS

Models. We test two 8B models in 4-bit AWQ quantization. Larger or unquantized models may recover differently, and Fragile Thoughts [20] finds that scale partly protects against perturbations, though not for every error type.

Planted errors. Our errors are substituted values of the right type, not mistakes the model made itself. Natural errors may be correlated with item difficulty in ways planted ones are not. Type matching uses coarse spaCy tags, and in the audit only about half of the planted values were the right fine-grained type, with 41% judged plausible by the LLM rater. The position effect is strongest among the cleanest values (Section VI-B), and it holds on plausible values alone. Absolute recovery rates, however, are somewhat higher for implausible values, so they overstate recovery from realistic errors.

Position and remaining steps. On linear chains, position, remaining steps and chain length are tied together (k + remaining = K). Branching items separate the two, but only for a minority of items (42–114 per model and mode). Four-hop items rarely pass eligibility without thinking (33 and 15 linear items), so the matched-remaining contrasts are underpowered. Branching items also differ from linear ones in ease, which our fit comparison controls for only through chain shape.

Selection. Eligibility keeps items that a model answers correctly along MuSiQue's decomposition and cannot answer closed-book (8–18% of attempted items). Results therefore describe errors on solvable, gold-aligned items. Different modes are eligible on different items; the shared-item analysis covers only 83 linear items.

Thinking-mode prefill. With the hop summary prefilled inside the think block, Qwen3 stops deliberating: continuations are about 110 tokens, and clean twins succeed at only 68–86% of intermediate hops. Our thinking-mode comparison is therefore between non-thinking generation and a thinking format with little deliberation after the error. Clean-twin conditioning further restricts it to items the model can finish without deliberating. [TBD: update after think_trace.]

Measurement. Each cell has one sample, so item difficulty is binary. Exact match counts answers at a different granularity as wrong. Explicit and silent recovery labels are rule-based and were not validated by hand. The thinking-mode cap of 4,096 tokens excluded 3% of baselines.

Data. MuSiQue is built from English Wikipedia, which the models have likely seen in training. The closed-book filter removes only items a model answers from memory.

VIII. Conclusion

We planted errors at exact hop positions in models' own multi-hop reasoning and measured how often the models recover. In both models and both modes, late intermediate errors are recovered less often than early ones. This late-stage fragility holds even when the answer-producing hop is excluded; that hop is a separate case, almost never recovered without thinking. For Qwen3 it is explained by the number of steps left after the error rather than by position itself; for Llama the evidence is mixed. Thinking raises recovery without flattening the position effect, and recovery is almost always silent. For verification in multi-step pipelines, these results suggest checking late steps, and the final step above all, rather than the first. For studies that prefill reasoning models, they show that where the prefix is placed decides whether the model deliberates at all. [TBD: revise the thinking sentence after think_trace.]

References

[1] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] A. von Recum, L. Girrbach, and Z. Akata, "Are reasoning LLMs robust to interventions on their chain-of-thought?" in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2602.07470.

[3] R. Jia and D. Mu, "From decorative to load-bearing: Task difficulty shapes the causal role of chain-of-thought," 2026, arXiv:2609.25366.

[4] D. Zhang, Y. Wu, Y. Sun, J. Yang, N. Yang, J. Zhu, M. Xin, and B. Tian, "Not all errors are created equal: ASCoT addresses late-stage fragility in efficient LLM reasoning," 2025, arXiv:2508.05282.

[5] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[6] Qwen Team, "Qwen3 technical report," 2025, arXiv:2505.09388.

[7] Llama Team, AI @ Meta, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[8] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of EMNLP*, 2018.

[9] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proceedings of COLING*, 2020.

[10] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in *Findings of EMNLP*, 2023.

[11] J. Yun, J. Kwon, and Y. Kim, "CRiT-QA: Evaluating multi-hop reasoning with counterfactual chains and distractor traps," in *Proceedings of LREC*, 2026, arXiv:2607.10562.

[12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, arXiv:2501.12948.

[14] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] G. Tyen, H. Mansoor, V. Cărbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them given the error location," in *Findings of ACL*, 2024.

[16] T. Lanham, A. Chen, A. Radhakrishnan *et al.*, "Measuring faithfulness in chain-of-thought reasoning," 2023, arXiv:2307.13702.

[17] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] Y. Chen, J. Benton, A. Radhakrishnan *et al.*, "Reasoning models don't always say what they think," 2025, arXiv:2505.05410.

[19] E. Yee, A. Li, C. Tang, Y. H. Jung, R. Paturi, and L. Bergen, "Dissociation of faithful and unfaithful reasoning in LLMs," 2024, arXiv:2405.15092.

[20] A. V. Aravindan and M. Kejriwal, "Fragile thoughts: How large language models handle chain-of-thought perturbations," 2026, arXiv:2603.03332.

[21] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[22] X. Chen, R. A. Chi, X. Wang, and D. Zhou, "Premise order matters in reasoning with large language models," in *Proceedings of ICML*, 2024.

[23] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *Proceedings of ICML*, 2023.

[24] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of EMNLP*, 2016.

[25] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020.

[26] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.